Projeto DevOps na Nuvem

Objetivo deste laboratório

Ao final dos quatro encontros, a sua dupla terá construído, com as próprias mãos, o **ciclo completo que leva um código do computador até uma aplicação funcionando na nuvem**, e saberá explicar cada etapa dele. É o mesmo caminho que equipes de desenvolvimento seguem no mercado, em escala reduzida e com ferramentas gratuitas.

Na prática, você sai com:

- um repositório no GitHub com histórico, ramos e pull requests, e um site publicado na nuvem pelo GitHub Pages (encontro 1);
- uma API em Flask, com testes automáticos, empacotada em um contêiner Docker que você mesmo construiu (encontro 2);
- um pipeline de integração contínua no GitHub Actions que testa o código e constrói a imagem a cada alteração (encontro 3);
- imagens versionadas publicadas no ghcr.io, uma página de status publicada automaticamente e a experiência de resolver uma falha com rollback (encontro 4).

Mais importante do que os comandos é o que eles ensinam. Ao final, você deve saber explicar:

- por que versionar o código e trabalhar com pull requests;
- o que diferencia IaaS, PaaS e SaaS, e onde o GitHub Pages se encaixa;
- por que um contêiner resolve o "funciona na minha máquina";
- por que testar e construir de forma automática a cada alteração;
- como identificar uma versão com problema e voltar para a que funcionava.

Cada encontro termina com um questionário individual, e em 30/11 a dupla demonstra o ciclo completo em 5 minutos.

Pré-requisitos

O roteiro foi escrito para quem já tem a base abaixo. Confira tudo **no encontro 1**: se algum item não estiver pronto, resolva com a sua dupla ou procure o professor com antecedência.

Infraestrutura

- **Conta no GitHub**, gratuita, criada com um e-mail que você consegue abrir na hora (o GitHub pode pedir um código de confirmação). **Não cadastre cartão de crédito**: nada neste laboratório exige pagamento.
- **Navegador atualizado** (Chrome, Edge ou Firefox). Todo o trabalho acontece nele, então não há nada para instalar no computador.
- **Conexão estável com a internet**, no laboratório ou em casa. Em casa, qualquer computador com navegador serve, até um notebook simples.
- **Dupla formada** e com o nome e a variável de cada integrante anotados, como descrito em "Antes de começar".
- **Um caderno ou bloco de notas** para registrar previsões, medições e erros, pois vários passos pedem isso.

Conhecimento técnico

Você não precisa dominar nada disso. Basta reconhecer os conceitos e conseguir usar o básico.

Área	O que você deve saber	Como conferir
Git	O que é repositório, commit, ramo (branch) e a diferença entre `git add`, `git commit` e `git push`.	Você sabe explicar o que cada um desses três comandos faz.
Terminal Linux	Navegar e manipular arquivos: `cd`, `ls`, `pwd`, `mkdir`, `cat`.	Você cria uma pasta, entra nela e lista o conteúdo sem consultar nada.
Python	Ler um programa curto: `import`, funções, variáveis e `print`.	Você lê uma função de 5 linhas e diz o que ela devolve.
Redes e web	O que são IP, porta, servidor, requisição e resposta HTTP.	Você explica por que `localhost:5000` e `localhost:5001` são endereços diferentes.
Formatos de texto	Ler HTML simples e JSON (`{"chave": "valor"}`).	Você aponta o título em uma página HTML curta e o valor de uma chave em um JSON.

O que **não** é pré-requisito: Docker, Flask, GitHub Actions, YAML e computação em nuvem. Tudo isso é apresentado no próprio roteiro, passo a passo.

Teste de prontidão (10 minutos)

Faça no dia do encontro 1:

1. Entre em github.com com a sua conta e confirme que o login funciona.
2. Responda no caderno, sem consultar: "o que acontece com os meus arquivos quando eu rodo `git add`, depois `git commit` e depois `git push`?"
3. Responda no caderno: "se um programa está escutando na porta 5000 de um computador, o que preciso saber para acessá-lo de outro?"

Se travar em algum ponto, não desista: revise o conceito com a sua dupla antes do encontro. As dificuldades de ambiente que surgirem durante as aulas estão na tabela de problemas comuns de cada encontro.

Antes de começar

1. Crie a sua conta gratuita no GitHub (cada aluno, a sua) e confirme o e-mail. **Não cadastre cartão de crédito.**
2. Combine a dupla e decidam quem será o aluno A e quem será o aluno B (tabela de papéis, abaixo). Anote o nome de usuário do GitHub de cada um.
3. Anote a **variação da dupla**, que o professor informa (tabela de variação, mais abaixo). Você usa esses três valores em todos os encontros.
4. Use um computador com navegador atualizado. Nada precisa ser instalado: o Git, o Python e o Docker vêm no ambiente de trabalho, o Codespaces, explicado mais abaixo.

Papéis da dupla. Cada aluno tem a sua conta no GitHub, mas o repositório é um só e fica na conta de um deles. Nos passos do roteiro, "(aluno A)" e "(aluno B)" dizem quem executa; "(os dois)" quer dizer que cada um faz no seu próprio Codespace.

Papel	Quem é	O que faz	Conta usada
Aluno A	Dono do repositório	Cria o repositório `projeto-devops-NOME_DUPLA` na própria conta, adiciona o aluno B como colaborador, ativa o Pages, revisa e faz o merge dos pull requests	A conta do aluno A
Aluno B	Colaborador	Aceita o convite, cria as branches e abre os pull requests	A conta do aluno B
Os dois	A dupla toda	Abrem, cada um, o seu Codespace no repositório do aluno A, fazem commits e `git push` e anotam no caderno	Cada um, a sua

O repositório fica sempre na conta do aluno A, do primeiro ao último encontro. Dica: seja o aluno A quem já tem o e-mail do GitHub confirmado, para não travar o início.

Valor	O que é	Exemplo
`NOME_DUPLA`	Nome único da dupla	dupla-07
`MENSAGEM`	Frase que aparece no seu site e na sua API	Olá da dupla 07
`PORTA`	Número da porta da sua API	5007

Dois combinados valem para o projeto inteiro:

- O repositório da dupla é **público**. No plano gratuito do GitHub, o Pages só publica repositórios públicos.
- Cada aluno faz commits com a **própria conta**. O histórico de commits mostra quem fez o quê.

O ambiente de trabalho: o GitHub Codespaces

Em todos os encontros você programa no **GitHub Codespaces**, um computador Linux na nuvem que abre dentro do navegador, já ligado ao repositório da sua dupla.

O que é. O GitHub cria para você uma máquina com editor de código, terminal e as ferramentas do projeto (Git, Python e Docker). Você acessa tudo por uma janela do navegador, e nada é instalado no computador do laboratório.

Para que serve. Para ter o mesmo ambiente de desenvolvimento em qualquer computador. É o que equipes de software usam para que uma pessoa nova comece a trabalhar sem gastar um dia instalando programas.

Por que estamos usando.

1. Todos trabalham no mesmo ambiente, então os comandos do roteiro funcionam igual para a turma toda. O "funciona na minha máquina" causado por diferença de instalação deixa de existir, e é um problema que o encontro 2 vai atacar de outro jeito.
2. Não é preciso instalar nada nem ter permissão de administrador nos computadores do laboratório.

3. O seu trabalho fica na nuvem. Se você trocar de computador, basta abrir o Codespace de novo.

Como abrir o seu Codespace (a cada encontro).

1. Entre no GitHub e abra o repositório da dupla. O aluno B só consegue depois de aceitar o convite de colaborador.
2. Clique no botão verde **Code**, abra a aba **Codespaces** e clique em **Create codespace on main**. Nos encontros seguintes, abra o Codespace que já existe na lista, em vez de criar outro.
3. Espere o ambiente carregar. Você verá a lista de arquivos à esquerda, o editor no centro e o terminal embaixo. Se o terminal não aparecer, use **Terminal > New Terminal**.

Cada aluno abre o **próprio** Codespace no mesmo repositório. Os dois se sincronizam como em uma equipe real, com `git push` e `git pull`.

Criar arquivos. No painel **Explorer**, clique no ícone de novo arquivo e digite o caminho completo, por exemplo `site/index.html`. A pasta é criada junto.

Git sem senha. O Codespace já entra no GitHub por você para o repositório de onde ele foi criado. Não é preciso digitar senha nem token.

Ver a sua página ou a sua API no navegador. Quando um programa seu passa a escutar uma porta, o Codespace a mostra na aba **PORTS** e oferece abri-la no navegador. A porta é privada: só você a enxerga, com a sua conta.

Salvar o trabalho. O que está no Codespace fica guardado enquanto ele existir, mas se ele for apagado, o que não foi enviado ao GitHub se perde. Codespaces parados são apagados sozinhos depois de 30 dias. Por isso, faça `git push` ao fim de cada passo importante.

Parar o Codespace ao sair. Fechar a aba do navegador **não** para o Codespace: ele continua ligado, gastando a sua cota, até ficar 30 minutos sem uso. Ao terminar, procure a opção **Stop current codespace** no menu do Codespaces (canto inferior esquerdo da janela) ou use a lista em github.com/codespaces.

Cota e cartão. A conta gratuita inclui 120 horas de computação por mês, o que basta para este projeto se você parar o Codespace ao sair. **Não cadastre cartão de crédito.** Sem cartão, se a cota acabasse, o Codespace deixaria de abrir até o mês seguinte, mas nada seria cobrado. Se isso acontecer, avise o professor.

Sintoma	Causa provável	O que fazer
O Codespace não abre ou não deixa criar	Cota do mês esgotada, ou a rede bloqueou o acesso	Avise o professor; não cadastre cartão
A aba PORTS não mostra a porta	O programa não está rodando, ou escuta em outra porta	Confira se o programa está ativo e use **Add port** com o seu número
O terminal sumiu	O painel foi fechado	Use **Terminal > New Terminal**
O Git reclama que não sabe quem você é, no primeiro commit	A identidade do Git não está configurada	Rode `git config --global user.name "Seu Nome"` e `git config --global user.email "seu-email@exemplo.com"`

Como funciona cada encontro

Você mantém **um único repositório por dupla**, que cresce a cada encontro e no fim mostra o ciclo completo de DevOps: versionar, empacotar, testar, publicar, falhar e voltar atrás.

Data	Encontro	Valor
26/10	1. Nuvem e versionamento	2,0
9/11	2. Contêineres	2,0
16/11	3. Integração contínua	2,0
23/11	4. Entrega contínua e operação	2,0
30/11	Apresentação da dupla	2,0

Todos os encontros têm 200 minutos (2 aulas de 100) e seguem o mesmo ritmo em cada passo:

1. **Preveja.** Antes de executar, escreva no caderno o que você acha que vai acontecer.
2. **Execute.** Rode o comando ou faça a mudança.
3. **Confira.** Compare com o que você previu. Quando errar a previsão, anote o que aprendeu: essa anotação vale mais que o acerto.

Os trechos marcados como **Checkpoint** dizem como saber que o passo deu certo. Se o checkpoint não bater, não avance.

No final da atividade há um **questionário**. As perguntas dependem do repositório da sua dupla (commits, nomes de arquivos, valores da sua variação), então deixe abertos o repositório, o site e a aba Actions na hora de responder.

O que você apresenta no final

No dia **30/11**, cada dupla apresenta por cerca de **5 minutos**. O professor vai até o computador da dupla, então **não precisa de slides**: vocês mostram o próprio projeto funcionando, no navegador e no Codespace, e os dois alunos falam.

Onde está cada encontro

Cada encontro e a apresentação têm a sua aba, na lista à esquerda. Abra a aba do dia e siga os passos na ordem.

- Encontro 1, 26/10, nuvem e versionamento
- Encontro 2, 9/11, contêineres
- Encontro 3, 16/11, integração contínua
- Encontro 4, 23/11, entrega contínua e operação
- Apresentação, 30/11, Apresentação